

Трансляция презентации (во время очных лекций)

<https://clck.ru/3D3Efj>



При просмотре презентации в PDF для отображения анимаций на слайдах необходимо использовать Acrobat Reader, KDE Okular, PDF-XChange, Foxit Reader, браузер Firefox. Для браузеров на движке Chrome (Edge, Яндекс, Опера, ...) необходимо использовать [PDF.js](#) с опцией «Enable active content (JavaScript) in PDFs».



Промышленное программирование

Тема 2

Разработка модульных программ на C++

Гаврилов Андрей Геннадьевич

кандидат технических наук, доцент

Кафедра Информационных технологий и вычислительных систем
МГТУ «СТАНКИН»

Обновлено: 13 мая 2026 г.

План лекции

1 Разделение кода на .h и .cpp файлы

2 Статические библиотеки

3 Динамические библиотеки

Напишем математическую программу

main.cpp

```
1 //include iostream cmath tuple exception clocale print
2
3 std::tuple<double,double> quadratic_equation(double a, double b, double c)
4 {
5     double D = b*b-4*a*c;
6     if (D>0)
7         return std::make_tuple( (-b + sqrt(D))/2/a , ( -b - sqrt(D))/2/a );
8     else
9         throw std::exception("D < 0 error!");
10 }
11
12 int main()
13 {
14     setlocale(LC_ALL, "Russian");
15     int a,b,c;
16     std::cin>>a>>b>>c;
17     auto [X1,X2] = quadratic_equation(a,b,c);
18     std::println("x1={} x2={}",X1, X2);
19     return 0;
20 }
```

Соберём и запустим её

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2)
2 project("quadratic equation")
3 set(CMAKE_CXX_STANDARD 23) #Нам нужна современная версия языка для std::println
4 add_executable(qe main.cpp)
```

```
PS ... > cmake -B build
-- Build files have been written to: .../build
PS ... > cmake --build build
qe.vcxproj - > .../build/Debug/qe.exe
1 -5 6    ⇐ Вводим коэф. уравнения
x1=3 x2=2 ⇐ Все работает!
```

Раздел 1

Разделение кода на .h и .cpp файлы

Основные форматы файлов

Заголовочные файлы (с расширением .h, .hpp, .hxx) —

это файлы, которые содержат объявления (декларации) функций, классов, переменных, констант и других сущностей, но не содержат их реализацию.

Файлы кода (с расширением .cpp, .cc, .cxx) —

это файлы, которые содержат реализацию (определения) функций, методов классов и других сущностей, объявленных в заголовочных файлах.

func.h

```
1 void foo(); //прототип
2
3 class Stankin //описание класса
4 {           //и его функций
5     int students_count = 0;
6     public:
7     void bar();
8 }
```

func.cpp

```
1 #include<func.h>
2 void foo() //реализация foo()
3 {
4     std::println("fooooo");
5 }
6 void Stankin::bar() //реализация
7 {                 //Stankin::bar()
8     std::println("baaarr");
9 }
```

Основные форматы файлов

Заголовочные файлы (с расширением .h, .hpp, .hxx) —

это файлы, которые содержат объявления (декларации) функций, классов, переменных, констант и других сущностей, но не содержат их реализацию.

Файлы кода (с расширением .cpp, .cc, .cxx) —

это файлы, которые содержат реализацию (определения) функций, методов классов и других сущностей, объявленных в заголовочных файлах.

func.h

```
1 void foo(); //прототип
2
3 class Stankin //описание класса
4 {           //и его функций
5     int students_count = 0;
6     public:
7     void bar();
8 }
```

func.cpp

```
1 #include<func.h>
2 void foo() //реализация foo()
3 {
4     std::println("fooooo");
5 }
6 void Stankin::bar() //реализация
7 {                 //Stankin::bar()
8     std::println("baaarr");
9 }
```

Сделаем новую библиотеку для исходной программы

mymath.h

```
1 #include <tuple>
2 std::tuple<double,double> quadratic_equation(double, double, double);
```

mymath.cpp

```
1 #include <cmath>
2 #include <exception>
3 #include "mymath.h"
4
5 std::tuple<double,double> quadratic_equation(double a, double b, double c)
6 {
7     double D = b*b-4*a*c;
8     if (D>0)
9         return std::make_tuple( (-b + sqrt(D))/2/a , ( -b - sqrt(D))/2/a );
10    else
11        throw std::exception("D < 0 error!");
12 }
```

Немного подправим main.cpp и CMakeLists.txt

main.cpp

```
1 //include <print> <iostream> <locale>
2 #include "mymath.h" // <-Наша библиотека
3 //cmath tuple exception больше не нужны тут
4
5 int main()
6 {
7     setlocale(LC_ALL, "Russian");
8     int a,b,c;
9     std::cin>>a>>b>>c;
10    auto [X1,X2] = quadratic_equation(a,b,c);
11    std::println("x1={} x2={}",X1, X2);
12
13    return 0;
14 }
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2)
2 project ("quadratic equation")
3 set (CMAKE_CXX_STANDARD 23)
4 add_executable(qe main.cpp mymath.cpp) #Добавляем сюда все cpp
```

Проверяем результат

```
PS ... > cmake -B build
...
-- Build files have been written to: .../build
```

```
PS ... > cmake --build build/
...
qe.vcxproj -> ... /build/Debug/qe.exe
```

```
PS ... > ./.build/Debug/qe.exe
1 -5 6
x1=3 x2=2  ← все работает!
```

```
PS ... > tree ./build/qe.dir/Debug /F
C:/.../BUILD/QE.DIR/DEBUG
|   main.obj           ← наши obj
|   mymath.obj         ← наши obj
|   ...
...
```

Проверяем результат

```
PS ... > cmake -B build
...
-- Build files have been written to: .../build

PS ... > cmake --build build/
...
qe.vcxproj -> ... /build/Debug/qe.exe

PS ... > ./.build/Debug/qe.exe
1 -5 6
x1=3 x2=2  ← все работает!
```

```
PS ... > tree ./build/qe.dir/Debug /F
C:/.../BUILD/QE.DIR/DEBUG
|   main.obj           ← наши obj
|   mymath.obj         ← наши obj
|   ...
...
```

Проверяем результат

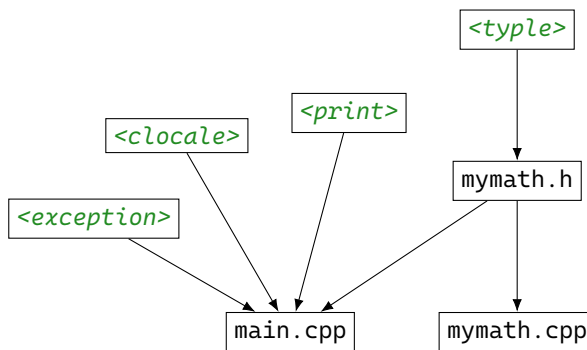
```
PS ... > cmake -B build
...
-- Build files have been written to: .../build

PS ... > cmake --build build/
...
qe.vcxproj -> ... /build/Debug/qe.exe

PS ... > ./build/Debug/qe.exe
1 -5 6
x1=3 x2=2  ← все работает!

PS ... > tree ./build/qe.dir/Debug /F
C:/.../BUILD/QE.DIR/DEBUG
|   main.obj           ← наши obj
|   mymath.obj         ← наши obj
|   ...
...
```

Схема включений



Усложним проект

mymath.h

```
1 #include <tuple>
2 std::tuple<double, double> quadratic_equation(double, double, double);
3
4 bool isEven(int n)
5 {
6     return n % 2 == 0;
7 }
```

main.cpp

```
1 ...
2 int main()
3 {
4     ...
5
6     std::println( "x1 is {}", isEven(X1)?"even":"odd" );
7
8     return 0;
9 }
```

Собираем

```
PS ..> cmake --build build/
Версия MSBuild 18.3.0-release-26070-10+3972042b7 для .NET Framework

Проверка источников на наличие зависимостей модуля...
mymath.cpp
main.cpp
Компиляция...
main.cpp
mymath.cpp
...
mymath.obj : error LNK2005: "bool __cdecl isEven(int)" (?isEven@@YA_NH@Z)
уже определен в main.obj
fatal error LNK1169: обнаружен многократно определенный
символ – один или более
```

Решение проблемы — inline

mymath.h

```
1 #include <tuple>
2 std::tuple<double, double> quadratic_equation(double, double, double);
3
4 inline bool isEven(int n)
5 {
6     return n % 2 == 0;
7 }
```

```
PS ... > cmake --build build/
...
qe.vcxproj -> C: ... /build/Debug/qe.exe
```

```
PS ... > ./build/Debug/qe.exe
1 -9 20
x1=5 x2=4
x1 is not odd ←←
```

Встроенные функции (inline)

Inline (подстановка) —

это указание компилятору, что код функции нужно вставить непосредственно в место вызова, вместо генерации инструкции вызова (call).

Основные цели

- Оптимизация скорости — убираются накладные расходы на вызов функции (создание стекового кадра, передача аргументов, возврат)
- Использование в заголовочных файлах — позволяет определять функции в .h файлах без ошибок множественного определения

```
1 // Без inline
2 int add(int a, int b) {
3     return a + b;
4 }
5
6 int main() {
7     int x = add(5, 3);
8     // Здесь будет инструкция CALL
9     return 0;
10 }
```

```
1 // C inline
2 inline int add(int a, int b) {
3     return a + b;
4 }
5
6 int main() {
7     int x = add(5, 3);
8     // Компилятор заменит на: int x = 5 + 3;
9     return 0;
10 }
```

Добавим константу в библиотеку

mymath.h

```
1 #include <tuple>
2 std::tuple<double, double> quadratic_equation(double, double, double);
3
4 inline bool isEven(int n) {
5     return n % 2 == 0;
6 }
7
8 //не будет работать!!
9 double half_PI = 1.5707963268;
```

Варианты:

```
1 //1. будет
2 const double half_PI = 1.5707963268;
3 //2. будет с++11
4 constexpr double half_PI = 1.5707963268;
5 //3. будет даже так! с++17
6 inline double half_PI = 1.5707963268;
7 //4. будет, но не стоит
8 static double half_PI = 1.5707963268;
```

Добавим константу в библиотеку

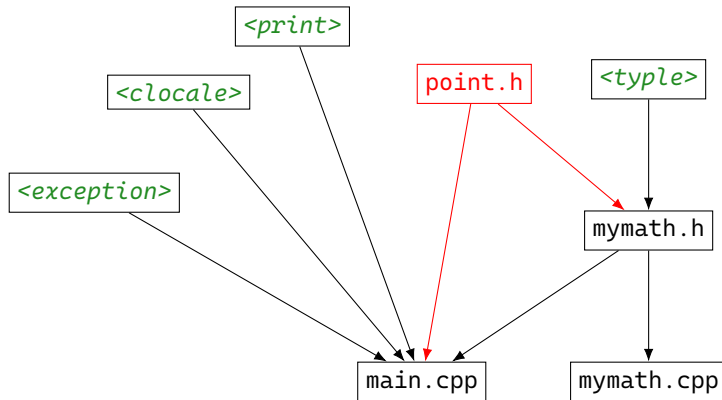
mymath.h

```
1 #include <tuple>
2 std::tuple<double,double> quadratic_equation(double, double, double);
3
4 inline bool isEven(int n) {
5     return n % 2 == 0;
6 }
7
8 //не будет работать!!
9 double half_PI = 1.5707963268;
```

Варианты:

```
1 //1. будет
2 const double half_PI = 1.5707963268;
3 //2. будет с++11
4 constexpr double half_PI = 1.5707963268;
5 //3. будет даже так! с++17
6 inline double half_PI = 1.5707963268;
7 //4. будет, но не стоит
8 static double half_PI = 1.5707963268;
```

Добавим новую библиотеку



Point.h

```
1 struct Point
2 {
3     double x;
4     double y;
5 };
```

Собираем с новой библиотекой

```
PS ...> cmake --build build/
Версия MSBuild 18.3.0-release-26070-10+3972042b7 для .NET Framework

Проверка источников на наличие зависимостей модуля...
mymath.cpp
main.cpp
Компиляция...
mymath.cpp
main.cpp
C:/.../point.h(1,8):
error C2011: Point: переопределение типа "struct"
[C:/.../build/qe.vcxproj]
(компиляция исходного файла "..\main.cpp")
C:/.../point.h(1,8):
см. объявление "Point"
```

Стражи заголовков

Стражи заголовков (Include Guards) —

конструкция препроцессора, защищающая от повторного включения одного и того же заголовочного файла. Предотвращают множественные определения функций и классов, которые вызывают ошибки компиляции.

Два способа реализации:

Через директиву `#pragma once`

Point.h

```
1 #pragma once //просто,  
2 //но не стандартно  
3 struct Point  
4 {  
5     double x;  
6     double y;  
7 };
```

Через конструкцию `#ifndef ... #endif`

Point.h

```
1 #ifndef POINT_H //сложнее, но всегда работает  
2 #define POINT_H //нужно придумать такое имя,  
3 //которое никогда не повторится  
4 struct Point  
5 {  
6     double x;  
7     double y;  
8 };  
9  
10 #endif //POINT_H
```

Стражи заголовков

Стражи заголовков (Include Guards) —

конструкция препроцессора, защищающая от повторного включения одного и того же заголовочного файла. Предотвращают множественные определения функций и классов, которые вызывают ошибки компиляции.

Два способа реализации:

Через директиву `#pragma once`

Point.h

```
1 #pragma once //просто,  
2 //но не стандартно  
3 struct Point  
4 {  
5     double x;  
6     double y;  
7 };
```

Через конструкцию `#ifndef ... #endif`

Point.h

```
1 #ifndef POINT_H //сложнее, но всегда работает  
2 #define POINT_H //нужно придумать такое имя,  
3 //которое никогда не повторится  
4 struct Point  
5 {  
6     double x;  
7     double y;  
8 };  
9  
10 #endif //POINT_H
```

Стражи заголовков

Стражи заголовков (Include Guards) —

конструкция препроцессора, защищающая от повторного включения одного и того же заголовочного файла. Предотвращают множественные определения функций и классов, которые вызывают ошибки компиляции.

Два способа реализации:

Через директиву `#pragma once`

Point.h

```
1 #pragma once //просто,  
2 //но не стандартно  
3 struct Point  
4 {  
5     double x;  
6     double y;  
7 };
```

Через конструкцию `#ifndef ... #endif`

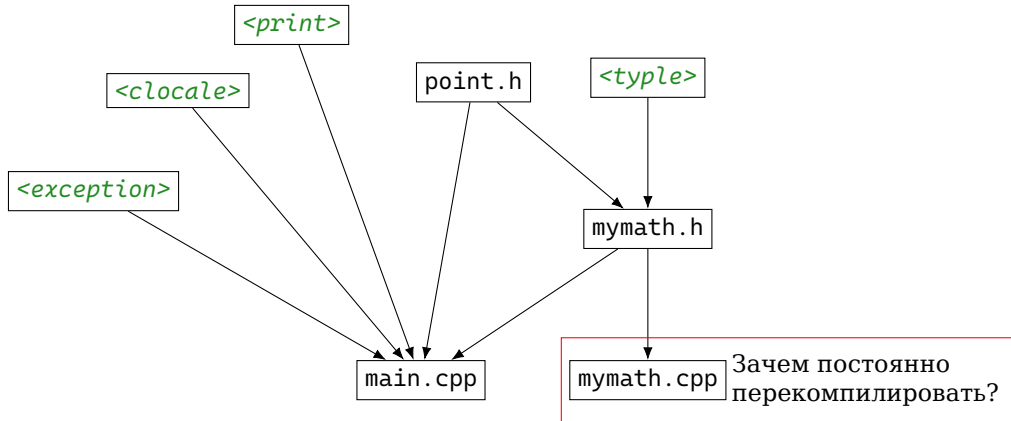
Point.h

```
1 #ifndef POINT_H //сложнее, но всегда работает  
2 #define POINT_H //нужно придумать такое имя,  
3 //которое никогда не повторится  
4 struct Point  
5 {  
6     double x;  
7     double y;  
8 };  
9  
10 #endif //POINT_H
```

Статические библиотеки

Статическая библиотека (static library) —

это набор скомпилированных объектных файлов (.o или .obj), объединенных в один архивный файл (с расширением .lib в Windows или .a в Linux/macOS), который связывается с программой на этапе компоновки (линковки).



Команды Сmake для работы с библиотеками

Объявление библиотеки

```
add_library(<name> [<type>] [EXCLUDE_FROM_ALL] <sources> ... )
```

type:

STATIC — статическая библиотека

SHARED — динамическая библиотека

MODULE — модуль

Подключение библиотеки

```
target_link_libraries(<target> ... <item> ... ... )
```

```
target_link_libraries(<target>  
    {INTERFACE|PUBLIC|PRIVATE} <item> ...  
    [{INTERFACE|PUBLIC|PRIVATE} <item> ... ] ... )
```

target — к чему подключаем (программа или библиотека)

item — что подключаем (библиотека)

Создадим библиотеку mymath

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2)
2 project("quadratic equation")
3 set(CMAKE_CXX_STANDARD 23)
4
5 #Создаем библиотеку
6 add_library(mymath mymath.cpp)
7 #Создаем саму программу
8 add_executable(qe main.cpp )
9 #Подключаем
10 target_link_libraries(qe mymath)
```

```
PS ...> cmake -B build ; cmake --build build/
...
mymath.vcxproj -> ... /build/Debug/mymath.lib  ⇐ библиотека!
qe.vcxproj -> ... /build/Debug/qe.exe  ⇐ программа!

PS ...> ./.build/Debug/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd  ⇐ все работает!
```

Создадим библиотеку mymath

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2)
2 project("quadratic equation")
3 set(CMAKE_CXX_STANDARD 23)
4
5 #Создаем библиотеку
6 add_library(mymath mymath.cpp)
7 #Создаем саму программу
8 add_executable(qe main.cpp )
9 #Подключаем
10 target_link_libraries(qe mymath)
```

```
PS ... > cmake -B build ; cmake --build build/
```

...

```
mymath.vcxproj -> ... /build/Debug/mymath.lib ← библиотека!
```

```
qe.vcxproj -> ... /build/Debug/qe.exe ← программа!
```

```
PS ... > ./build/Debug/qe.exe
```

```
1 -5 6
```

```
x1=3 x2=2x1 is not odd ← все работает!
```

Проведём эксперимент

```
PS ... > mkdir new_folder
```

```
Directory: C:/...
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d----	12.03.2026 1:18		new_folder

```
PS ... > cp ./build/Debug/qe.exe ./new_folder/
```

```
PS ... > dir ./new_folder/
```

```
Directory: C:/.../new_folder
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a---	12.03.2026 1:05	1106432	qe.exe

```
PS ... > ./new_folder/qe.exe
```

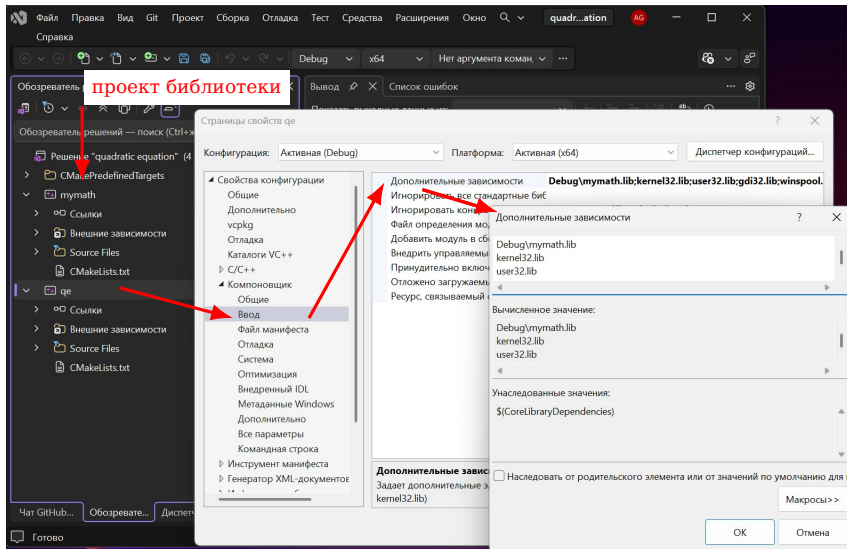
```
1 -5 6
```

```
x1=3 x2=2x1 is not odd
```

← все работает!

```
PS ... > rm -r ./new_folder
```

Как это выглядит в IDE



Динамические библиотеки

Динамическая библиотека (разделяемая библиотека, shared library) —

это файл, содержащий скомпилированный код и данные, который загружается в память во время выполнения программы, а не включается непосредственно в исполняемый файл.

Основные отличия от статических библиотек:

- Линковка на этапе выполнения.
- Машинный код расположен в отдельном файле.
- Загружаются в память по потребности.
- Одна библиотека может многократно использоваться разными программами.
- Работает медленнее.
- Могут быть проблемы с совместимостью.

Создадим динамическую библиотеку mymath

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2)
2 project ("quadratic equation")
3 set (CMAKE_CXX_STANDARD 23)
4
5 #Создаем динамическую библиотеку
6 add_library(mymath SHARED mymath.cpp) #добавили SHARED
7 #Создаем саму программу
8 add_executable(qe main.cpp )
9 #Подключаем
10 target_link_libraries(qe mymath)
```

Создадим динамическую библиотеку mymath

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 4.2)
2 project ("quadratic equation")
3 set (CMAKE_CXX_STANDARD 23)
4
5 #Создаем динамическую библиотеку
6 add_library(mymath SHARED mymath.cpp) #добавили SHARED
7 #Создаем саму программу
8 add_executable(qe main.cpp )
9 #Подключаем
10 target_link_libraries(qe mymath)
```

Подправим код библиотеки для dll

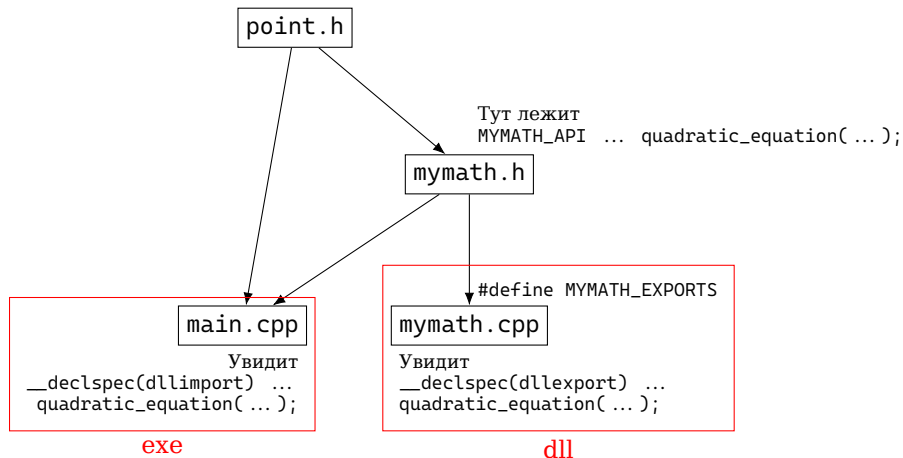
mymath.h

```
1 #include <tuple>
2 #include "point.h"
3
4 // Этот макрос будет экспортировать функции из DLL
5 #ifdef MYMATH_EXPORTS
6     #define MYMATH_API __declspec(dllexport) //функция экспортируется
7 #else
8     #define MYMATH_API __declspec(dllimport) //функция импортируется
9 #endif
10
11 MYMATH_API std::tuple<double,double> quadratic_equation(double, double, double);
12
13 MYMATH_API inline bool isEven(int n) {
14     return n % 2 == 0;
15 }
```

mymath.cpp

```
1 ...
2 #define MYMATH_EXPORTS //Добавляем макрос
3 #include "mymath.h"
4 ...
```

Схема проекта с динамической библиотекой



MYMATH_API $\Rightarrow$ __declspec(dllimport) в exe
MYMATH_API $\Rightarrow$ __declspec(dllexport) в dll

Результат

```
PS ... > rm -r ./build/ -Force; cmake --build build/; cmake --build build/
...
Создается библиотека .../build/Debug/mymath.lib и объект lib!! o_0
.../build/Debug/mymath.exp mymath.vcxproj -> .../build/Debug/mymath.dll
...
qe.vcxproj -> .../build/Debug/qe.exe
PS ... > ./build/Debug/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
PS ... > mkdir new_folder
PS ... > cp ./build/Debug/qe.exe ./new_folder/
PS ... > ./new_folder/qe.exe
PS ... >                                     ⇐= ничего не работает =(
PS ... > cp ./build/Debug/mymath.dll ./new_folder/
PS ... > ./new_folder/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
```

Результат

```
PS ... > rm -r ./build/ -Force; cmake --build build/; cmake --build build/
...
Создается библиотека .../build/Debug/mymath.lib и объект lib!! o_0
.../build/Debug/mymath.exp mymath.vcxproj -> .../build/Debug/mymath.dll
...
qe.vcxproj -> .../build/Debug/qe.exe
PS ... > ./build/Debug/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
PS ... > mkdir new_folder
PS ... > cp ./build/Debug/qe.exe ./new_folder/
PS ... > ./new_folder/qe.exe
PS ... >                                     ⇐ ничего не работает =(
PS ... > cp ./build/Debug/mymath.dll ./new_folder/
PS ... > ./new_folder/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
```

Результат

```
PS ... > rm -r ./build/ -Force; cmake --build build/; cmake --build build/
...
Создается библиотека .../build/Debug/mymath.lib и объект lib!! o_0
.../build/Debug/mymath.exp mymath.vcxproj -> .../build/Debug/mymath.dll
...
qe.vcxproj -> .../build/Debug/qe.exe
PS ... > ./build/Debug/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
PS ... > mkdir new_folder
PS ... > cp ./build/Debug/qe.exe ./new_folder/
PS ... > ./new_folder/qe.exe
PS ... >                                     ⇐ ничего не работает =(
PS ... > cp ./build/Debug/mymath.dll ./new_folder/
PS ... > ./new_folder/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
```